Tema 03: Tipos de datos abstractos en C++: Clases

METODOLOGÍA DE LA PROGRAMACIÓN

GRADO EN INGENIERÍA INFORMÁTICA

DECSAI

UNIVERSITAS GRANATENSIS · 1531 · PLUS ULTRA

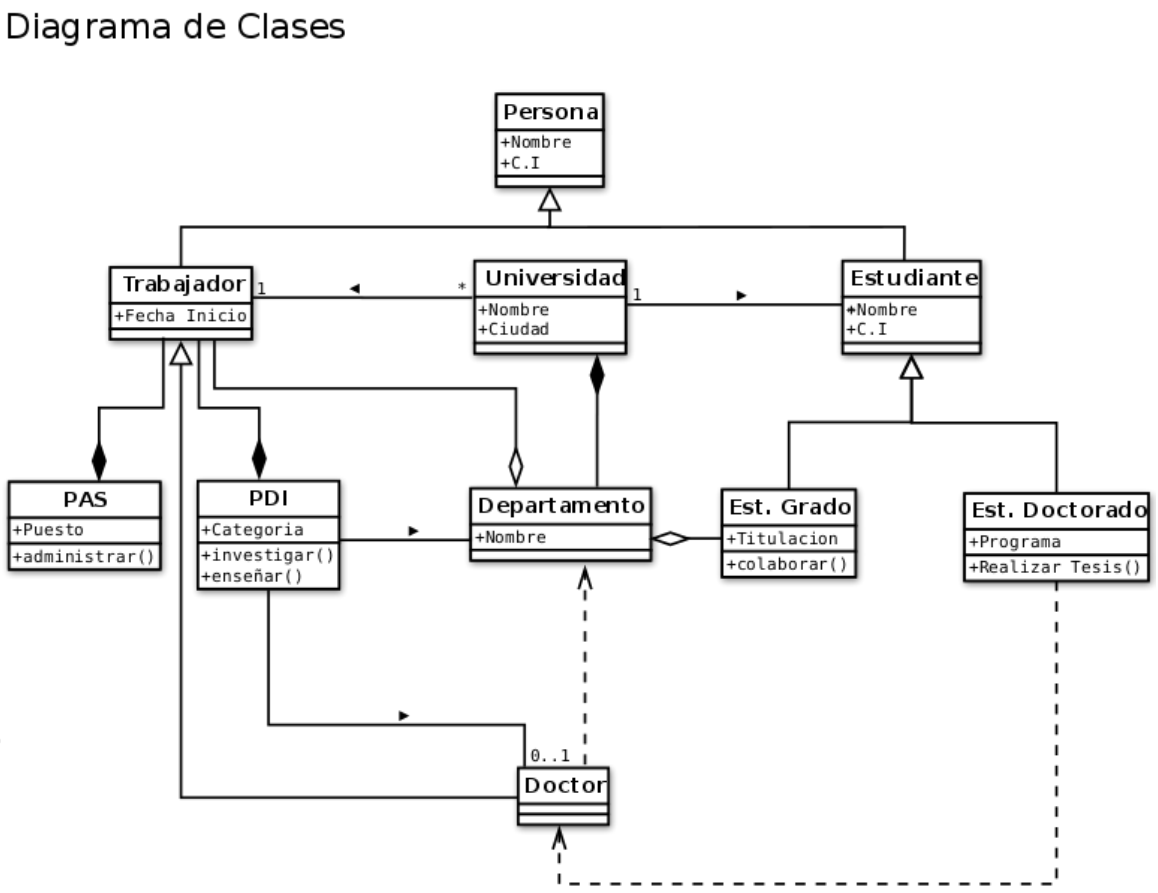
Índice

- Abstracción y diseño de clases: atributos y métodos.
- Constructores y destructores.
- Asignación en clases que gestionan memoria dinámica.
- Sobrecarga de operadores.

Atributos y métodos

Clases en C++

- Las clases en C++ nos permiten:
  - Crear nuestros propios tipos de datos.
  - Trabajar en un paradigma orientado a objetos.
- Con cada clase, podemos crear uno o más objetos que interactúan entre sí.
- Una clase puede tener como dato miembro un objeto de otra clase.



Atributos y métodos

DECSAI

Tipos de enfoques para programación:

Programación estructurada.

- Usar únicamente las estructuras de control vistas. Prohibir uso de goto y similares.

Programación modular.

Programación procedural.

- Tenemos un conjunto de funciones accesibles desde cualquier parte a las que vamos llamando.

Programación orientada a objetos.

- Se usan clases. Cada clase representa un objeto. Un objeto representa datos y funciones asociadas a los datos.

Atributos y métodos

DECSAI

Clase vs objeto

Clase:

- Es el tipo de dato.
- Especifica la estructura del objeto y su comportamiento.

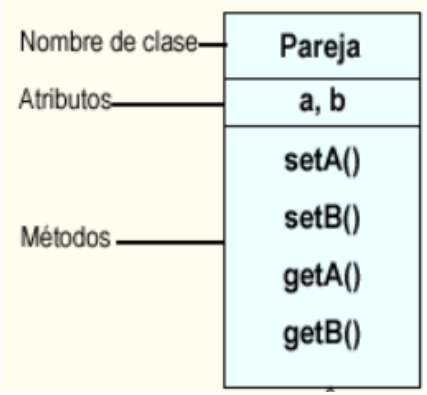
Objeto:

- Es el dato, la variable.
- Tiene unos valores específicos.
- Se dice que un objeto es “una instancia” de una clase.

Atributos y métodos

Encapsulación

- La encapsulación es el mecanismo de modularización utilizado en programación dirigida a objetos. La idea consiste en aunar datos y comportamiento en un mismo módulo.
- En el mismo componente, se encapsulan datos y funciones (métodos).



Atributos y métodos

DECSAI

Componentes de una clase

- **Datos miembro.**
  - Variables que se crearán en cada objeto de la clase. Cada objeto puede tener sus datos miembro con distintos valores.
  - Si se declaran datos como estáticos, son datos compartidos por todos los objetos.
  - Estado del objeto: Valores de sus datos miembro en un momento específico.

- **Métodos miembro.**
  - Determinan el comportamiento del objeto. Es decir, lo que ese objeto puede hacer.
  - Todos los objetos tienen los mismos métodos.

Atributos y métodos

DECSAI

Datos públicos y privados

Cada dato y método puede indicarse como público o como privado.

Dato o método público: Puede accederse desde fuera del objeto.

Dato o método privado: Sólo puede accederse al método o al dato desde métodos de dentro de la clase.

Constructores y destructores

DECSAI

Recordatorio

- **Constructor**: Función que se ejecuta cada vez que se crea un objeto de la clase.
- **Destructor**: Función que se ejecuta cada vez que se destruye un objeto de la clase.
- **Constructor de copia**: Función que crea un nuevo objeto como copia de un objeto previamente creado.
- Si no especificamos cualquiera de estos 3 elementos se utiliza el constructor, destructor, o constructor de copia por defecto.
- Dependiendo del caso, puede ser necesario crear nuestros propios constructores, constructores de copia y destructores.

Constructores y destructores

Constructores, destructores y memoria dinámica

- Los constructores y destructores por defecto no funcionan adecuadamente cuando trabajamos con memoria dinámica.
- El constructor por defecto no inicializa el puntero ni reserva la memoria.
- El constructor de copia hace que un el puntero del objeto nuevo creado apunte al mismo espacio de memoria que el objeto origen.
- El destructor por defecto no libera el espacio reservado en memoria dinámica.
- Conclusión: En caso de usar memoria dinámica, hemos de definirlos nosotros.

Constructores y destructores

DECSAI

¿Cuándo se ejecuta el destructor?

- Respuesta rápida: Cuando la variable deja de existir.
- Ejemplos:
  - Cuando en una función tenemos un objeto como local, al acabar la función se destruye.
  - Los objetos declarados en el main se destruyen cuando el main termina de ejecutarse.

Constructores y destructores

DECSAI

Uso del constructor de copia

- Crea un objeto a partir de otro de la misma clase.

- Se define como:

```
<nombreClase>(const <nombreClase> & <objetoClaseOrigen>);
```

- Se usa como:

```
<nombreClase> <nuevoObjeto>(<objetoOrigenYaCreado>)
```

Constructores y destructores

Ejemplo de uso del constructor, destructor y constructor de copia

Poner todo a 0 es opcional

```
MatrizFloat::MatrizFloat(int filae, int cole)
{
    fils = filae;
    cols = cole;
    reservaMemoriaMatriz();
    for(int i = 0; i < fils;i++){
        for(int j = 0; j < cols;j++){
            m[i][j] = 0;
        }
    }
}

MatrizFloat::~MatrizFloat()
{
    liberaMemoria();
}
```

```
void MatrizFloat::reservaMemoriaMatriz(){
    m = new float * [fils];
    for(int i = 0; i < fils; i++){
        m[i] = new float[cols];
    }
}

void MatrizFloat::liberaMemoria(){
    for(int i = 0; i < fils; i++){
        delete [] m[i];
    }
}
```

Constructores y destructores

Ejemplo de uso del constructor, destructor y constructor de copia

```
MatrizFloat::MatrizFloat(const MatrizFloat & a){
    fils = a.fils;
    cols = a.cols;
    reservaMemoriaMatriz();
    for(int i = 0; i < fils;i++){
        for(int j = 0; j < cols;j++){
            m[i][j] = a.m[i][j];
        }
    }
}
```

Constructor de copia

Uso del constructor de copia

```
MatrizFloat a(3,2);
MatrizFloat b(a);
```

```
#ifndef MATRIZFLOAT_H
#define MATRIZFLOAT_H


class MatrizFloat
{
    public:
        MatrizFloat(int filae, int cole);
        MatrizFloat(const MatrizFloat & a);
        ~MatrizFloat();
        void imprimeMatriz();
        void rellenaValores(float value, int filmentry, int colsmentry);
    private:
        void reservaMemoriaMatriz();
        void liberaMemoria();
        float ** m;
        int fils;
        int cols;
};

#endif // MATRIZFLOAT_H
```

Constructores y destructores

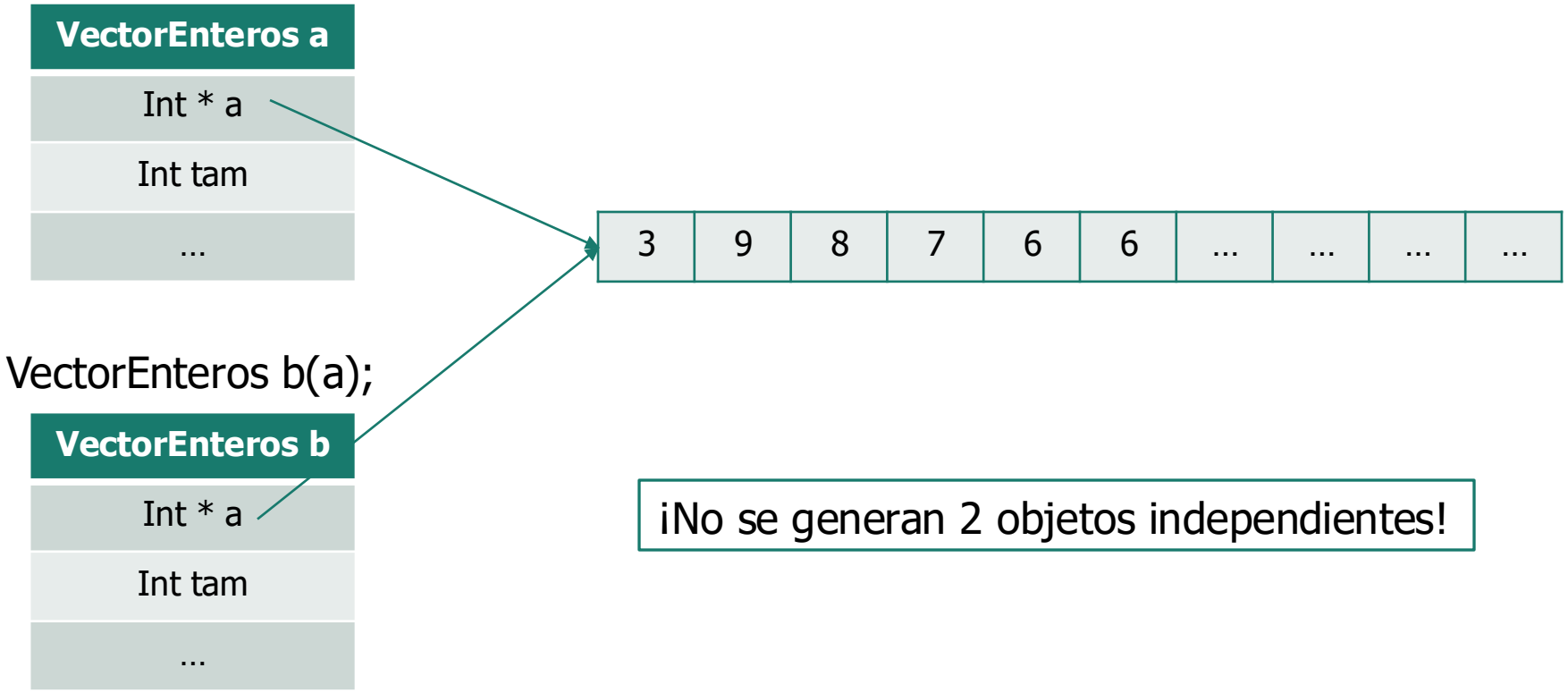
DECSAI

Detalles a tener en cuenta en los constructores

- Si usamos memoria dinámica, debemos de reservar el espacio necesario para almacenar la información e inicializar los punteros adecuadamente.
- En los destructores debemos de eliminar toda memoria reservada por el objeto. En caso de no hacerlo, dicha memoria queda reservada para siempre mermando los recursos del sistema.
- En el constructor de copia debemos de reservar espacio en el objeto nuevo y realizar una copia valor a valor de todos los elementos.
- Una vez hemos definido nuestros propios constructores, debemos asignar por nosotros mismos valores correctos a todos los datos miembro de la clase.

Constructores y destructores

Constructor de copia



¡No se generan 2 objetos independientes!

Referencias, punteros y clases

Uso de clase en memoria heap con punteros

```
class Persona{
    private:
    string nombre;
    string DNI;
    int edad;
    public:
        Persona(string DNIe, string nombree, int edade){
            DNI = DNIe;
            nombre = nombree;
            edad = edade;
        }
        //Las demás funciones
        void asignaNombre(string nuevonombre){
            nombre = nuevonombre;
        }
        void asignaEdad(int nuevaedad){
            edad = nuevaedad;
        }
        void asignaDNI(string nuevoDNI){
            DNI = nuevoDNI;
        }
        void MuestraDatos(){
            cout << DNI << " " << nombre << " " << edad << endl;
        }
};
```

```
int main(){
    //sin punteros
    Persona p("67876545V","Juan", 34);
    p.asignaNombre("Pedro");
    p.MuestraDatos();
    //con punteros
    Persona * puntp = new Persona("67876545V","Juan", 34);
    (*puntp).asignaNombre("Pedro");
    puntp->asignaEdad(5); //esto y lo de arriba es equivalente
    puntp->MuestraDatos();

}
```

Referencias, punteros y clases

DECSAI

Modificación de datos miembro mediante referencias

- Podemos utilizar el tipo de dato referencia para modificar valores miembro privados en objetos.

- Proceso:
  - Creamos una función que devuelve una referencia al valor que queremos modificar.
  - Modificamos ese valor utilizando la referencia.

- Ventaja: Podemos modificar el valor miembro sí y sólo si utilizamos una función especifica manteniendo, de esta manera, el valor miembro su carácter privado.

Referencias, punteros y clases

Modificación de datos miembro mediante referencias

- Ejemplo:

```
class Persona{
    private:
    string DNI;
    string nombre;
    int numeroHijos;

      ...

    int & modificaNumHijos(){
        return numeroHijos;
    }

    Persona(string DNIe, string nombree, int numeroHijose){
        DNI = DNIe;
        nombre = nombree;
        numeroHijos = numeroHijose;
    }
};
```

```
int main(){
    Persona jesus("76423478V","Jesus", 5);
    cout << jesus.numHijos() << " " << endl;
    jesus.modificaNumHijos() = 6;
    cout << jesus.numHijos() << " " << endl;
}
```

```
5
6

Process returned 0 (0x0)   execution time : 2.179 s
Press any key to continue.
```

Referencias, punteros y clases

DECSAI

Objetos con métodos const

- Los métodos const evitan que se modifiquen cualquier dato miembro de la clase.
- Es bueno utilizarlo en todos los métodos de la clase que devuelvan información pero no la modifiquen.
- También puede usarse para el cálculo de valores generados a partir de los datos miembro.
- Simplemente añadimos const al final de la declaración de la función:

```
<tipo> <nombreMétodo>(<parámetros>) const
```

Referencias, punteros y clases

Objetos con métodos const

Ejemplo:

```
class Persona{
    private:
    string DNI;
    string nombre;

    public:

    void setDNI(string elDNI){
        DNI = elDNI;
    }
    void setNombre(string elnombre){
        nombre = elnombre;
    }
    Persona(){
    }
    string verDNI() const{
        //setNombre("pedro"); //Esto da error
        return DNI;
    }
    string verNombre() const{
        return nombre;
    }
```

Referencias, punteros y clases

DECSAI

Objetos con métodos const

- Cuando pasamos como parámetro a una función un objeto como const, sólo podemos llamar a aquellos métodos que hayan sido definidos como const.
- Si lo pasamos como parámetro sin const podemos llamar a métodos const y no const.
- **Motivo**: Evitar la modificación de valores del objeto:
- Problema: Si no colocamos const delante de todos los métodos que no modifiquen los valores miembro del objeto no sirve de mucho.

Referencias, punteros y clases

Objetos con métodos const

```
void ImprimeValoresMetodo(const Persona & a){
    cout << a.verDNI() << " " << a.verNombre() << endl;
    //a.setNombre("Pedro"); ERROR, a es const
}

int main(){
    Persona jesus("56545665G","Jesus");
    ImprimeValoresMetodo(jesus);
}
```

- Si verDNI() fuera una función no const entonces no podría llamarse desde ImprimeValoresMetodo(const Persona) incluso aunque esta función no modificara a los valores miembro.

Referencias, punteros y clases

DECSAI

Métodos que devuelven referencias al objeto modificado

- A veces puede interesar que el método que modifica la clase devuelva el objeto modificado.
- Esto permite encadena llamadas a función lo que permite hacer el código más compacto.
- Dentro de la función, lo que hacemos es, una vez terminados los cómputos, devolvemos una referencia al propio objeto.

Referencias, punteros y clases

DECSAI

Ejemplo con la clase string

```
string cadena = "Holaaaaa";
cadena.replace(1,2,"mundo").replace(4,5,"todos").size();
cout << cadena << endl;
```

```
Hmuntodosaa

Process returned 0 (0x0)   execution time : 1.404 s
Press any key to continue.
```

Referencias, punteros y clases

DECSAI

Puntero this

- Dentro de los métodos en donde trabajamos, no tenemos acceso de forma directa al objeto con el que estamos trabajando, trabajamos directamente con sus datos miembro y funciones.

- Solución: Puntero this.

- This es un puntero que podemos utilizar en la implementación de cualquier método y que hace referencia al objeto con el que estamos trabajando.

Referencias, punteros y clases

Ejemplo de uso de this

```
class Par{
    int x=0;
    int y=0;
    public:
    int verX(){
        return x;
    }
    int verY(){
        return y;
    }

    void setPar(int x, int y){
        //no usar esto porque lleva a confusion
        this->x = x;
        this->y = y;
    }

    void imprime(){
        cout << this->x << " " << this->verY() << endl;
    }

    Par & aniade(Par newpar){
        x+=newpar.verX();
        this->y +=newpar.verY();
        return *this;
    }
};

int main(){
    Par p,q;
    p.setPar(4,5);
    q.setPar(2,2);
    q = p.aniade(q);
    p.imprime();
    q.imprime();
}
```

Imprime: 6 7
6 7

Referencias, punteros y clases

DECSAI

Puntero this

- En verdad, cuando en un método llamamos a otro método de la clase, en verdad estamos haciendo this->otrometodo().
- Cuando accedemos a los datos miembro también estamos haciendo:
  - this->dato_miembro
- Generalmente, el único sitio donde nos hace falta especificar el puntero this, va a ser:
  - Cuando queramos devolver una referencia al propio objeto. En el resto de casos, el uso de this se puede omitir.
  - Cuando queramos comprobar si un objeto que nos pasan al método es el mismo.

Referencias, punteros y clases

Comparación de 2 objetos para ver si son el mismo:

```
bool compara(Par & x){
    if (this == &x){
        return true;
    }else{
        return false;
    }
}
```

```
cout << p.compara(q) << " " << p.compara(p) << " " << p.compara(r) << endl;
```

Resultado: 0 1 0

Referencias, punteros y clases

DECSAI

Contexto constante vs contexto no constante

- **Contexto constante**.
  - No queremos que se modifique la referencia.
  - Tanto la referencia devuelta como el método deben ser constantes.

```
const clase & metodo(<parámetros>) const
```

- **Contexto no constante**.
  - Podemos modificar los datos miembro del objeto.
  - Tanto la función como la referencia aparecen sin constante.
  - Este contexto permite la modificación del objeto.

```
clase & metodo(<parámetros>)
```

Reserva de memoria para matrices en clases

DECSAI

Matrices con memoria dinámica

- Para tener un control sobre el acceso que se realiza a una matriz, podemos crear una clase que encapsule toda la mecánica relacionada con una matriz.

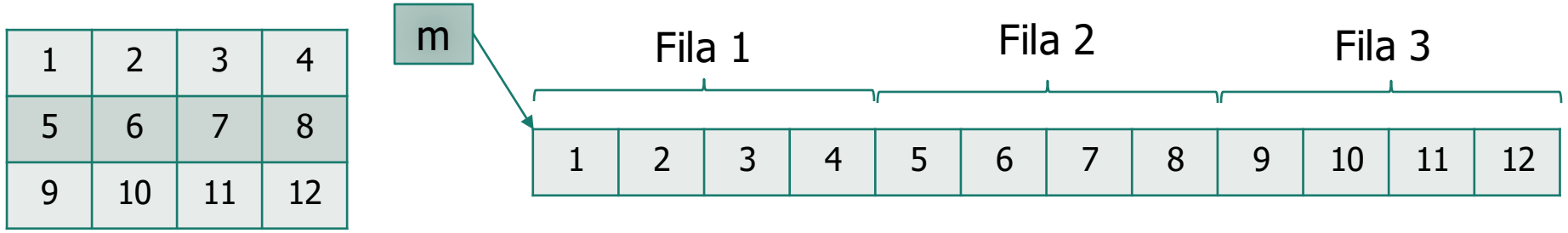
- Veremos 2 tipos de reserva para matriz dinámica con tamaño fijo de fila y columna:
  - **Matriz1D**: Reservamos un vector de datos en donde almacenamos todos los datos de la matriz.
  - **Matriz2D**: Usamos un puntero a punteros para reservar el espacio necesario.

- Detalles a tener en cuenta: Eficiencia de las operaciones que vayan a llevarse a cabo, memoria utilizada y si la matriz va a crecer y si lo hará por filas o por columnas.

Reserva de memoria para matrices en clases

Matriz 1D

- Utilizamos un vector en memoria para almacenar todos los datos de la matriz.



- Problema: No podemos utilizar directamente los operadores [][] para hacer el acceso.

Reserva de memoria para matrices en clases

DECSAI

Matriz1D: Datos miembro de la clase y funciones de acceso

```
class Matriz1D
{
    public:
        Matriz1D();
        Matriz1D(int fils, int cols);
        Matriz1D(int fils, int cols, int valor);
        Matriz1D(const Matriz1D & morig);
        ~Matriz1D();
        const int & at(int fils,int cols) const;
        int & at(int fils, int cols);
        void clear();


    private:
        int * m;
        int filas;
        int columnas;

};
```

```
int & Matriz1D::at(int f, int c){
    assert(0<=f && f < filas);
    assert(0<=c && c < columnas);
    return(m[columnas*f+c]);
}

const int & Matriz1D::at(int f, int c) const{
    assert(0<=f && f < filas);
    assert(0<=c && c < columnas);
    return(m[columnas*f+c]);
}
```

Reserva de memoria para matrices en clases

Matriz1D: Funciones de reserva y liberación de memoria

```
Matriz1D::Matriz1D()
{
    filas=columnas=0;
    m = nullptr;
}

Matriz1D::Matriz1D(int fils, int cols){
    filas = fils;
    columnas = cols;
    m = new int [fils*cols];
}

Matriz1D::Matriz1D(int fils, int cols, int valor){
    filas = fils;
    columnas = cols;
    m = new int [fils*cols];
    for(int i = 0; i < fils*cols;i++){
        m[i] = valor;
    }
}
```

```
Matriz1D::Matriz1D(const Matriz1D & morig){
    filas = morig.filas;
    columnas = morig.columnas;
     m = new int [filas*columnas];
      for(int i = 0; i < filas*columnas;i++){
        m[i] = morig.m[i];
    }
}

void Matriz1D::clear(){
     if(m != nullptr) delete [] m;
     filas=columnas=0;
     m = nullptr;
}

Matriz1D::~Matriz1D()
{
    if(m != nullptr) delete [] m;
}
```

Reserva de memoria para matrices en clases

DECSAI

Ventajas e inconvenientes de Matriz1D

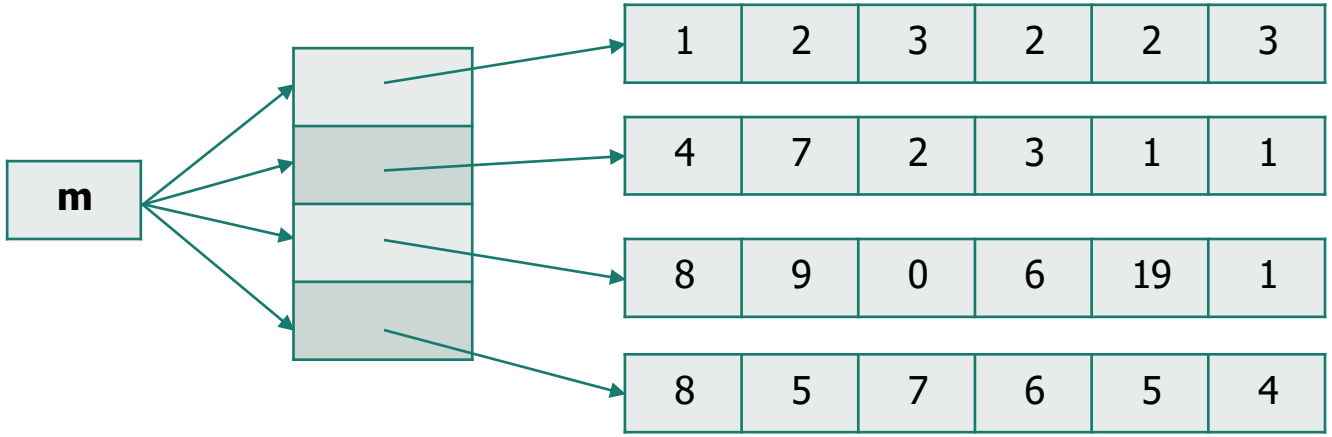
- No podemos usar directamente los operadores [][] para acceder a los datos.
- Es eficiente leer la matriz por filas.
- La reserva y liberación de memoria es sencilla.
- No utilizamos elementos complejos como un puntero a punteros.

Reserva de memoria para matrices en clases

Matriz2D

- La vimos en el tema 1.
- Utilizamos un puntero a punteros para almacenar la información.

1	2	3	2	2	3
4	7	2	3	1	1
8	9	0	6	19	1
8	5	7	6	5	4



- Puede accederse con [][].

Reserva de memoria para matrices en clases

Matriz2D: Datos miembro de la clase y funciones de acceso

```
class Matriz2D
{
        public:
        Matriz2D();
        Matriz2D(int fils, int cols);
        Matriz2D(int fils, int cols, int valor);
        Matriz2D(const Matriz2D & morig);
        ~Matriz2D();
        const int & at(int fils,int cols) const;
        int & at(int fils, int cols);
        void clear();

        private:
        int ** m;
        int filas;
        int columnas;
};
```

```
int & Matriz2D::at(int f, int c){
    assert(0<=f && f < filas);
    assert(0<=c && c < columnas);
    return(m[f][c]);
}

const int & Matriz2D::at(int f, int c) const{
    assert(0<=f && f < filas);
    assert(0<=c && c < columnas);
    return(m[f][c]);
}
```

Reserva de memoria para matrices en clases

DECSAI

Matriz2D: Funciones de reserva y liberación de memoria

```
Matriz2D::Matriz2D()
{
    filas=columnas=0;
    m = nullptr;
}

Matriz2D::Matriz2D(int fils, int cols){
    filas = fils;
    columnas = cols;
    m = new int* [fils];
    for(int i = 0; i < fils;i++){
        m[i] = new int [cols];
    }
}

Matriz2D::Matriz2D(int fils, int cols, int valor){
    filas = fils;
    columnas = cols;
    m = new int* [fils];
    for(int i = 0; i < fils;i++){
        m[i] = new int [cols];
    }
    for(int i = 0; i < fils;i++){
        for(int j = 0; j < cols;j++){
            m[i][j] = valor;
        }
    }
}
```

```
Matriz2D::Matriz2D(const Matriz2D & morig){
    filas = morig.filas;
    columnas = morig.columnas;
    m = new int* [filas];
    for(int i = 0; i < filas;i++){
        m[i] = new int [columnas];
    }
    for(int i = 0; i < filas;i++){
        for(int j = 0; j < columnas;j++){
            m[i][j] = morig.m[i][j];
        }
    }
}

void Matriz2D::clear(){
    if(m != nullptr){
        for(int i =0; i < filas; i++){
            delete [] m[i];
        }
        delete[] m;
    }
    filas=columnas=0;
    m = nullptr;
}
```

Reserva de memoria para matrices en clases

Matriz2D: Funciones de reserva y liberación de memoria

```
Matriz2D::~Matriz2D()
{
    if(m != nullptr){
        for(int i =0; i < filas; i++){
            delete [] m[i];
        }
        delete[] m;
    }
}
```

Reserva de memoria para matrices en clases

DECSAI

Ventajas e inconvenientes de Matriz2D

- Podemos utilizar [][] para acceder a los datos sin realizar sobrecarga.
- Es eficiente leer la matriz por filas.
- La reserva y liberación de memoria es algo más compleja.
- Utilizamos puntero a punteros para reservar la memoria.
- Tanto Matriz1D como Matriz2D se puede utilizar para que sea más eficiente por columnas si almacenamos en cada fila, una columna de la matriz y modificamos los operadores de acceso.
- En Matriz2D, podemos reservar nuevas filas si creamos un nuevo vector de punteros, asignamos los punteros ya existentes e inicializamos los nuevos.

Reserva de memoria para matrices en clases

DECSAI

RaggMatriz

- RaggMatriz es una matriz que tiene un tamaño distinto de elementos por filas.
- Una alternativa menos eficiente a una RaggMatriz es una matriz normal con un número de columnas igual a la que tenga la fila con más elementos. El resto de filas desperdicia espacio.
- Gracias a las funciones de gestión de memoria dinámica, podemos crear RaggMatrices que ocupen únicamente el espacio de memoria necesario para el almacenamiento de los datos.

Reserva de memoria para matrices en clases

RaggMatriz

- Al igual que con las matrices, podemos almacenar la información en un único vector o en un puntero a punteros donde cada puntero del vector de punteros apunta a una fila diferente.

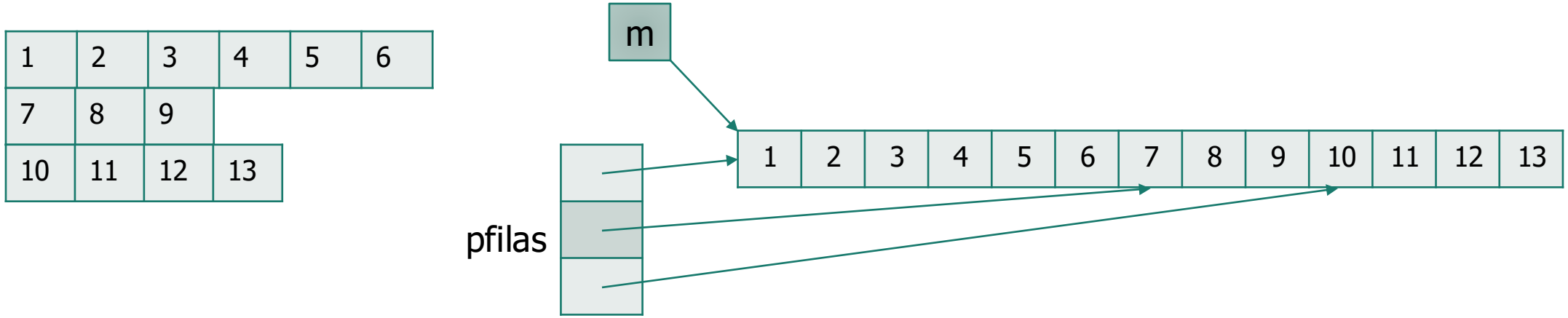
- **RaggMatriz1D**: Almacenamos en un único vector.
- **RaggMatriz2D**: Almacenamos en puntero a punteros.

1	2	3	4	5	6
7	8	9			
10	11	12	13		

Reserva de memoria para matrices en clases

RaggMatriz1D

- Los datos se almacenan en un bloque continuo de datos.
- Tenemos un array de punteros que apunta a cada fila de datos.



Reserva de memoria para matrices en clases

DECSAI

RaggMatriz1D: .h y funciones de acceso

```
class RaggMatriz1D
{
    public:
        RaggMatriz1D();
        //filas con igual tamanio
        RaggMatriz1D(int f,int c);
        //filas con diferente tamanio
        RaggMatriz1D(int f, int *c);
        int & at(int f, int c);
        const int & at(int f, int c)const;
        ~RaggMatriz1D();
        void clear();
        RaggMatriz1D(const RaggMatriz1D & orig);

    private:
        int * m;
        int ** pfilas;
        int filas;
        int total;

};
```

```
int & RaggMatriz1D::at(int f,int c){
    assert(0<=f && f < filas);
    if(f!=filas-1) assert(0<=c && c< pfilas[f+1]-pfilas[f]);
    else assert(0<=c && c< (m+total) -pfilas[f]);
    return pfilas[f][c];
}

const int & RaggMatriz1D::at(int f,int c)const {
    assert(0<=f && f < filas);
    if(f!=filas-1) assert(0<=c && c< pfilas[f+1]-pfilas[f]);
    else assert(0<=c && c< (m+total) -pfilas[f]);
    return pfilas[f][c];
}
```

Reserva de memoria para matrices en clases

RaggMatriz1D: Constructores

```
RaggMatriz1D::RaggMatriz1D()
{
    filas = total = 0;
    m = nullptr;
    pfilas = nullptr;
}

RaggMatriz1D::RaggMatriz1D(int f, int c){
    filas = f;
    total = f*c;
    m = new int [total];
    pfilas = new int*[filas];
    for(int i = 0; i < filas;i++){
        pfilas[i] = &m[i*c];
    }
}
```

```
RaggMatriz1D::RaggMatriz1D(int f, int * c){
    filas = f;
    total = 0;
    for(int i = 0; i < f; i++) total+=c[i];
    m = new int [total];
    pfilas = new int * [filas];
    int pos = 0;
    for(int i = 0; i < filas;i++){
        pfilas[i] = &m[pos];
        pos+=c[i];
    }
}

RaggMatriz1D::RaggMatriz1D(const RaggMatriz1D & orig){
    filas = orig.filas;
    total = orig.total;
    m = new int [total];
    pfilas = new int * [filas];
    for(int i = 0; i < total;i++){
        m[i] = orig.m[i];
    }
    for(int i = 0; i < filas;i++)
    pfilas[i] = &m[orig.pfilas[i]-orig.m];
}
```

Reserva de memoria para matrices en clases

DECSAI

RaggMatriz1D: Destructores

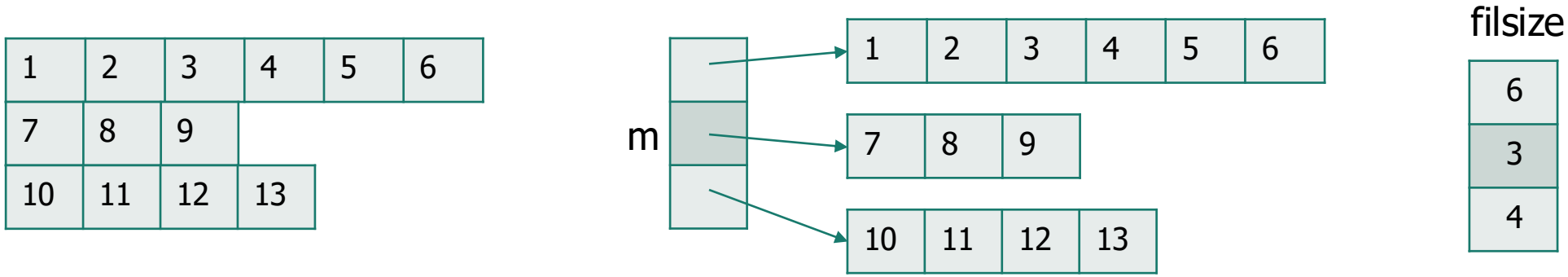
```
RaggMatriz1D::~RaggMatriz1D()
{
    if(m!=nullptr){
     delete[] m;
     delete [] pfilas;
    }
}
```

```
void RaggMatriz1D::clear(){
    if(m!=nullptr){
        delete[] m;
        delete [] pfilas;
    }
    filas = total = 0;
    m = nullptr;
    pfilas = nullptr;
}
```

Reserva de memoria para matrices en clases

RaggMatriz2D

- Necesitamos guardar el tamaño de cada columna en un vector.
- Los datos se almacenan guardando cada fila en un puntero distinto.



Reserva de memoria para matrices en clases

DECSAI

RaggMatriz2D: .h y acceso

```
class RaggMatriz2D
{
    public:
        RaggMatriz2D();
        //filas con igual tamanio
        RaggMatriz2D(int f,int c);
        //filas con diferente tamanio
        RaggMatriz2D(int f, int *c);
        int & at(int f, int c);
        const int & at(int f, int c)const;
        ~RaggMatriz2D();
        void clear();
        RaggMatriz2D(const RaggMatriz2D & orig);

    private:
        int ** m;
        int * tamfilas;
        int filas;
        int total;
};
```

```
int & RaggMatriz2D::at(int f,int c){
    assert(0 <=f && f < filas);
    assert(0 <=c && c < tamfilas[f]);
    return m[f][c];
}

const int & RaggMatriz2D::at(int f,int c)const {
    assert(0 <=f && f < filas);
    assert(0 <=c && c < tamfilas[f]);
    return m[f][c];
}
```

Reserva de memoria para matrices en clases

DECSAI

RaggMatriz2D: Constructores

```
RaggMatriz2D::RaggMatriz2D()
{
    filas = total = 0;
    m = nullptr;
    tamfilas = nullptr;
}

RaggMatriz2D::RaggMatriz2D(int f, int c){
    filas = f;
    total = f*c;
    tamfilas = new int [f];
    for(int i = 0; i < f; i++){
        tamfilas[i] =c;
    }
    m = new int * [filas];
    for(int i =0; i < filas;i++){
        m[i] = new int [tamfilas[i]];
    }
}
```

```
RaggMatriz2D::RaggMatriz2D(int f, int * c){
    filas = f;
    total = 0;
   tamfilas = new int [f];
   for(int i = 0 ; i < f;i++){
        tamfilas[i] = c[i];
        total+=tamfilas[i];
   }
   m = new int*[filas];
   for(int i = 0; i < filas; i++){
    m[i] = new int [tamfilas[i]];
   }
}
```

Reserva de memoria para matrices en clases

DECSAI

RaggMatriz2D: Constructor de copia

```
RaggMatriz2D::RaggMatriz2D(const RaggMatriz2D & orig){
    filas = orig.filas;
    total = orig.total;
    tamfilas = new int [filas];
    for(int i = 0; i < filas; i++){
        tamfilas[i] = orig.tamfilas[i];
    }
    m = new int *[filas];
    for(int i = 0; i < filas;i++){
        m[i] = new int [tamfilas[i]];
        for(int j = 0; j < tamfilas[i];j++){
            m[i][j] = orig.m[i][j];
        }
    }
}
```

Reserva de memoria para matrices en clases

RaggMatriz2D: Destructores

```
RaggMatriz2D::~RaggMatriz2D()
{
    if(m!=nullptr){
        for(int i = 0; i < filas;i++)
            delete[] m[i];
        delete[] m;
        delete[] tamfilas;
    }
}
```

```
void RaggMatriz2D::clear(){
    if(m!=nullptr){
        for(int i = 0; i < filas;i++)
            delete[] m[i];
        delete[] m;
        delete[] tamfilas;
    }
    filas = total = 0;
    m = nullptr;
    tamfilas = nullptr;
}
```

Reserva de memoria para matrices en clases

DECSAI

RaggMatriz1D vs RaggMatriz2D

- Ambas ocupan un espacio muy similar en memoria.
- RaggMatriz1D minimiza el número de elementos ya que no necesita almacenar el tamaño de cada fila.
- RaggMatriz2D es más fácil de manejar porque almacena el tamaño de cada fila.
- RaggMatriz1D utiliza el puntero de la siguiente final para determinar cuando hemos terminado de leer una fila.
- Ambas opciones son buenas y permiten un correcto manejo de la estructura.

Sobrecarga de operadores

DECSAI

Motivación

- Tanto las funciones como los métodos de una clase pueden sobrecargarse.
- Podemos utilizar el mismo nombre de función o método para diferentes parámetros de entrada.
- Algunas ventajas: Aumento de la legibilidad, aplicación de la misma función a diferentes tipos, etc.
- Una función siempre es más compleja de definir y usar que un operador.
- Los operadores también se pueden sobrecargar.

Sobrecarga de operadores

DECSAI

Ventajas de la sobrecarga de operadores

- Mayor legibilidad en el código.
- Mayor comodidad al programar.
- Aplicación de las operaciones ya implementadas para tipos simples en tipos complejos creados por nosotros.
- Aprovechamiento del conjunto de operadores proporcionados en C++ para nuestros tipos.

Sobrecarga de operadores

¿Qué podemos sobrecargar?

- Algunas sobrecargas de operadores ya existentes:
  - Los operadores + y – están sobrecargados de forma que funcionan con todos los tipos numéricos (int, float, double, etc.).
  - El operador asignación (=) también está sobrecargado y funciona diferente según estemos hablando de tipos básicos u objetos de clase.

- Los siguientes operadores se pueden sobrecargar:

Operadores que pueden sobrecargarse

+	-	*	/	%	^	&	|
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	|=	<<	>>	>>=
<<=	==	!=	<=	>=	&&	||	++
–	->*	,	->	[]	()	new	delete
new[]	delete[]						

Sobrecarga de operadores

DECSAI

Restricciones en la sobrecarga de un operador

- No se puede cambiar el número de parámetros que utiliza un operador.
  - Ejemplo: No podemos crear un operador + que utilice siempre 3 valores. Aunque si podemos sumar 2 valores y añadirle un tercero: 5+6+7.
- Estos operadores no pueden sobrecargarse:

.	.*	::	?:

- Los operadores

&	*	+	-

tienen versiones unitarias que sí pueden sobrecargarse.
- Sobrecargar un operador no sobrecargable: error de sintaxis.
- No se puede modificar la precedencia de un operador ni su asociatividad.

Sobrecarga de operadores

DECSAI

Restricciones en la sobrecarga de un operador

- No podemos crear nuevos operadores. Por ejemplo, no podemos crear un operador que sea **.
- No podemos modificar cómo funciona un operador con los tipos de datos predefinidos.
- Al menos un argumento utilizado en la sobrecarga del operador debe de ser de un tipo predefinido por el usuario.
- Sobrecargar el operador + no implica que += quede sobrecargado, hay que implementarlo aparte.

Sobrecarga de operadores

DECSAI

Hay 2 maneras de sobrecargar un operador

- **Como método miembro**.
  - Sólo puede utilizarse cuando el operando que aparece más a la izquierda es del tipo de la clase.

- **Como función no miembro**.
  - Necesario cuando el operando que aparece más a la izquierda no es del tipo de la clase.
  - Opcional cuando el operando que aparece más a la izquierda es del tipo de la clase.
  - Cuando el operador se implementa como función no miembro el programa es ligeramente más ilegible con lo que es preferible implementarlo como método miembro siempre que se pudiera.

Sobrecarga de operadores

DECSAI

Funciones friend

- Las funciones friend son un tipo de funciones especiales que pueden acceder a los datos miembro de una clase a pesar de ser funciones externas a la clase.
- Aunque las funciones friend rompen con la protección privada de los datos de una clase, suelen utilizarse en la sobrecarga de operadores por eficiencia y sencillez.
- Para indicar a la clase que utilice el operador definido fuera de la clase, tenemos que declararlo dentro de la clase (ahora veremos ejemplos).

Sobrecarga de operadores

DECSAI

Los operadores de flujo << y >>

- Ambos operadores, para operar correctamente, necesitan un objeto ostream y istream a su izquierda respectivamente.
- Por tanto, deben implementarse como funciones externas a la clase.
- Su uso ha sido visto en la asignatura con cin y cout, objetos istream y ostream que se encargan de recoger información de teclado y mostrar datos en la pantalla.
- Veamos a continuación, como crear nuestros propios operadores >> y << para mostrar y rellenar información en objetos.

Sobrecarga de operadores

Ejemplo de uso de los operadores de flujo

```
class Persona{
    friend ostream& operator<<(ostream&,const Persona &);
    friend istream& operator>>(istream&, Persona &);
    private:
    string nombre;
    string DNI;
    int edad;
    public:
        //Las demás funciones
};

ostream& operator<<(ostream & out, const Persona & p){
    out << p.DNI << ", " << p.nombre << " edad:" << p.edad << endl;;
    return out;
}

istream& operator >>(istream& in, Persona & p){
    in >> p.DNI >> p.nombre >> p.edad;
    return in;
}
```

Para permitir el acceso a los datos privados se declaran aquí

Uso

```
int main(){
    Persona p;
    cin >> p;
    cout << p;
}
```

```
76345898B, Pedro edad:23

Process returned 0 (0x0)   execution time : 17.602 s
Press any key to continue.
```

Definidas fuera de la clase

Sobrecarga de operadores

DECSAI

Funcionamiento del flujo cin

- cin >> p hace que el compilador llame a operator>>(cin,p)
- “in” pasa a ser un alias para cin y p es el objeto donde almacenar la información.
- La función realiza el proceso según le indicamos.
- El objeto p, al estar como valor por referencia, queda rellenado con la información.
- Return devuelve el flujo modificado en la función.

Sobrecarga de operadores

DECSAI

Más detalles sobre la definición de los operadores de flujo

- En << se pasa el objeto como const ya que el operador no debe modificar el objeto a mostrar.
- En >>, como el objeto sí es modificado, se pasa con una referencia.
- La declaración de la función como friend es opcional, se suele hacer así por eficiencia en el acceso a los datos.
- Si la función no fuera friend, habría que usar las funciones públicas de la clase para acceder a los datos.

Sobrecarga de operadores

DECSAI

Número de parámetros en los operadores de sobrecarga

- Dependiendo de si definimos el operador dentro o fuera de la clase, el número de parámetros del operador cambia.
- **Si la definimos fuera**: Debemos meter como parámetros todos los elementos involucrados. Ejemplo: Operador de flujo.
- **Si la definimos dentro**: El primer parámetro es el objeto de la clase donde se está definiendo la función. El resto de los valores implicados deben ir como parámetros.
- Ejemplo → operadores unarios: 0 parámetros si se definen dentro de la clase y 1 si se define fuera.

Sobrecarga de operadores

DECSAI

Sobrecarga con operadores unarios

- Puede hacerse como función miembro de la clase.
- La llamada se hace sobre el objeto donde se ha definido la función.
- Al definirse dentro de la clase, no necesita parámetros.
- Definición:

```
<tipodevuelto> operator<operador>();
```

Sobrecarga de operadores

DECSAI

Ejemplo de sobrecarga de operador unario

- Operador ! en Persona para ver si el DNI es del tamaño correcto.

Definición del operador:

```
bool operator!()const{
    if(DNI.size() != 9) return true;
    else return false;
}
```

Uso del operador:

```
int main(){
    Persona p;
    cin >> p;
    if (!p) cout << "ERROR EN DNI" << endl;
    cout << p;
}
```

Sobrecarga de operadores

DECSAI

Sobrecarga de operadores binarios

- Si lo definimos como función miembro:
  - El primero operando es el objeto donde se está definiendo.
  - El segundo operando se pasa como parámetro.
- Si lo definimos como función no miembro:
  - Los 2 operandos se pasan como parámetro a la función.
- Siempre que se pueda es mejor definirlo como función miembro ya que mejora la claridad del código.

Sobrecarga de operadores

Ejemplo de sobrecarga de operador binaria

- Operador += que sume al objeto un número de años al año de la persona.

```
Persona & operator+=(int valor){
    edad = edad+valor;
    return *this;
}
```

- Ejemplo de llamada: m+=5;
- *this devuelve un puntero al propio objeto ya modificado.

Sobrecarga de operadores

DECSAI

Sobrecarga de operadores con 2 parámetros

- La sobrecarga de operadores tales como +, -, *, / requieren de 2 parámetros si se implementa como función miembro.
- El resultado se asigna en el objeto desde donde se llama al operador.
- EJEMPLO: Podemos crear un operador suma que sirve para sumar 2 clases vector creadas por nosotros.
- En los archivos VectorSobrecargado.cpp y VectorSobrecargado.h encontramos varios ejemplos de otros operadores que podemos sobrecargar.

Sobrecarga de operadores

Ejemplo de sobrecarga de + en clase vector

Como usamos .h y .cpp ponemos operador de ámbito

```
VectorSobrecargado & VectorSobrecargado::operator+(VectorSobrecargado & a){
    VectorSobrecargado * resultado = new VectorSobrecargado(a.tam);
    if(a.tam ==tam){
            for(int i = 0; i < a.tam;i++){
                (*resultado)[i] = a[i] + v[i];
            }
    }
    return (*resultado);
}
```

Sobrecarga de operadores

Ejemplo 2 de sobrecarga de + en clase vector

```
VectorSobrecargado operator+(VectorSobrecargado &b, VectorSobrecargado & a){
    VectorSobrecargado resultado(a.tam);
    if(a.tam == b.tam){
            for(int i = 0; i < a.tam;i++){
                resultado[i] = b.v[i] + a.v[i];
            }
    }
    return resultado;
}
```

- Este ejemplo declara el objeto en memoria estática, aunque la memoria del vector si está en memoria dinámica.

Sobrecarga de operadores

Ejemplo de sobrecarga de operador de acceso

- El operador devuelve una referencia que puede utilizarse para modificar la posición de memoria especificada.

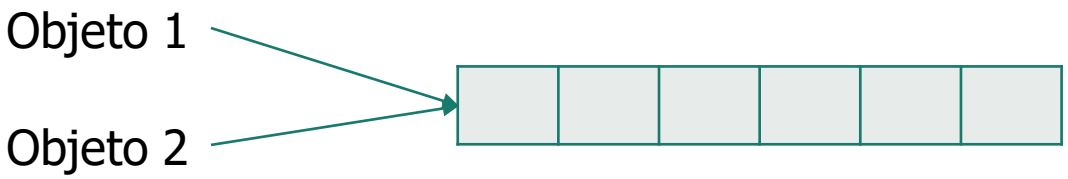
```
int & VectorSobrecargado::operator[](int pos){
        if(pos >=0 && pos < tam)
        return v[pos];
        else {
            cerr << "Error al acceder a la posicion" << endl;
            exit(-1);
        }
}
```

Sobrecarga de operadores

Operador de asignación por defecto vs operación de asignación definido por nosotros

- Todas las clases tienen un operador de asignación definido por defecto.
- Aunque por lo general va bien, cuando tratamos con clases que contienen vectores puede dar problemas.
- El vector de asignación por defecto hace que el puntero del objeto que estamos asignando apunte al mismo vector que el del objeto asignado.

¡¡Los objetos no son independientes entre sí!!



Sobrecarga de operadores

Operador de asignación designado por nosotros

- Aparte de asignar el mismo valor a todos los valores miembro del objeto debemos copiar los valores de un vector en el espacio del otro.

```
const VectorSobrecargado & VectorSobrecargado::operator=(const VectorSobrecargado & a){
    tam = a.tam;
    v = new int [tam];
    for(int i = 0; i < tam;i++){
        v[i] = a.v[i];
    }
    return (*this) ;//habilita x = y = z
}
```